



sAlfer Lab

Joint lab on Safety and Security of AI

Dai bit al calcolatore

Rappresentazione ed elaborazione dell'informazione

Luca Didaci | Giorgio Fumera

luca.didaci@unica.it | giorgio.fumera@unica.it



UNICA

UNIVERSITÀ
DEGLI STUDI
DI CAGLIARI

La rappresentazione dell'informazione

- Rappresentazione dei numeri interi
- Il bit, il byte e i suoi multipli
- Rappresentazione dei caratteri

L'informazione

Gli odierni sistemi per l'elaborazione dell'informazione sono formati da elementi che possono assumere due soli stati:

0 o 1.

È possibile rappresentare l'informazione utilizzando tali elementi?

...

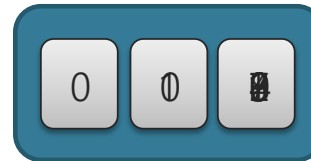
1	0	1	1	0	1	1	0	1	0	1
---	---	---	---	---	---	---	---	---	---	---

Esaminiamo la possibilità di *rappresentare numeri interi*. In seguito vedremo come è possibile rappresentare informazioni più complesse: testi, immagini, musica, ecc.

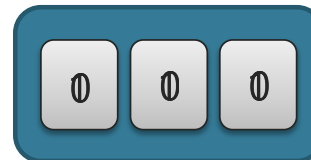
Vedremo anche come è possibile *elaborare*, e non solo *rappresentare*, tali informazioni.

Rappresentazione dei numeri interi

- Esempio: contatore con dieci simboli ...



- ... e contatore con due simboli



Rappresentazione dei numeri interi

Posso stabilire una corrispondenza **arbitraria** tra sequenze di $\{0,1\}$ e i numeri interi. L'importante è che la corrispondenza sia univoca.

Sequenza di bit	Numero intero
000100	0
000001	1
000010	2
000011	3
001011	4
000101	5
000110	6
000111	7
001000	8
001001	9
001010	10
000000	11
...	...

Rappresentazione dei numeri interi

Esistono però corrispondenze 'migliori'(*) di altre. In seguito sarà chiaro perché utilizzeremo questa corrispondenza e non la precedente.

Sequenza di bit	Numero intero
000000	0
000001	1
000010	2
000011	3
000100	4
000101	5
000110	6
000111	7
001000	8
001001	9
001010	10
001011	11
...	...

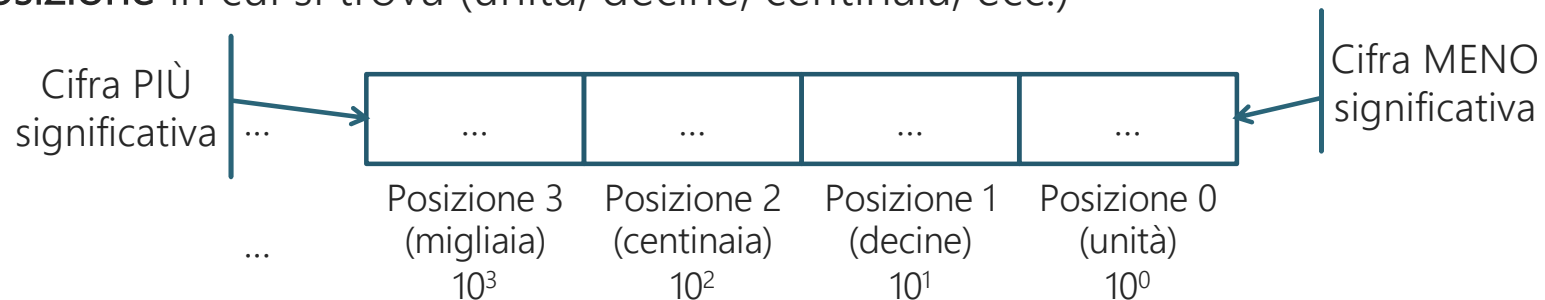
(*) esistono altre corrispondenze utili, ad es. [https://it.wikipedia.org/wiki/Codice di Hamming](https://it.wikipedia.org/wiki/Codice_di_Hamming)

Rappresentazione dei numeri interi

Sistema di numerazione posizionale in base 10

(sistema numerico decimale)

- Alfabeto di simboli (cifre) $\{0,1,2,3,4,5,6,7,8,9\}$
- Sistema posizionale: ogni cifra assume un **significato diverso** (peso) a seconda della **posizione** in cui si trova (unità, decine, centinaia, ecc.)



Ho 10 simboli e il 'peso' associato a ciascuna posizione è dato dalle potenze di 10: 10^0 per la prima posizione, 10^1 per la seconda, 10^2 per la terza e così via

Rappresentazione dei numeri interi

Se chiamiamo a_i la cifra in i -esima posizione, l'insieme di simboli

a_3	a_2	a_1	a_0
10^3	10^2	10^1	10^0

rappresenta il numero

$$\begin{aligned} & a_3 \cdot 10^3 + a_2 \cdot 10^2 + a_1 \cdot 10^1 + a_0 \cdot 10^0 = \\ & = a_3 \cdot 1000 + a_2 \cdot 100 + a_1 \cdot 10 + a_0 \cdot 1 \end{aligned}$$

Esempio

Consideriamo l'insieme di simboli 1971

$$\begin{aligned} & 1 \cdot 10^3 + 9 \cdot 10^2 + 7 \cdot 10^1 + 1 \cdot 10^0 = \\ & = 1 \cdot 1000 + 9 \cdot 100 + 7 \cdot 10 + 1 \cdot 1 = 1971 \end{aligned}$$

Rappresentazione dei numeri interi

Numerazione romana: sistema additivo *non posizionale*

M=1000

C=100

L=50

X=10

V=5

I=1

ciascun simbolo
rappresenta sempre uno
specifico valore

1971 = MCMLXXI

1971 =	M	C M	L	X	X	I
	1000	900	50	10	10	1

Rappresentazione dei numeri interi

Esempio: vantaggio della numerazione posizionale

$$\begin{array}{r} 1971 + \\ 1970 = \\ \hline 3941 \end{array}$$

$$\begin{array}{r} \text{MCMLXXI} + \\ \text{MCMLXX} = \\ \hline \text{----} \\ \dots \end{array}$$

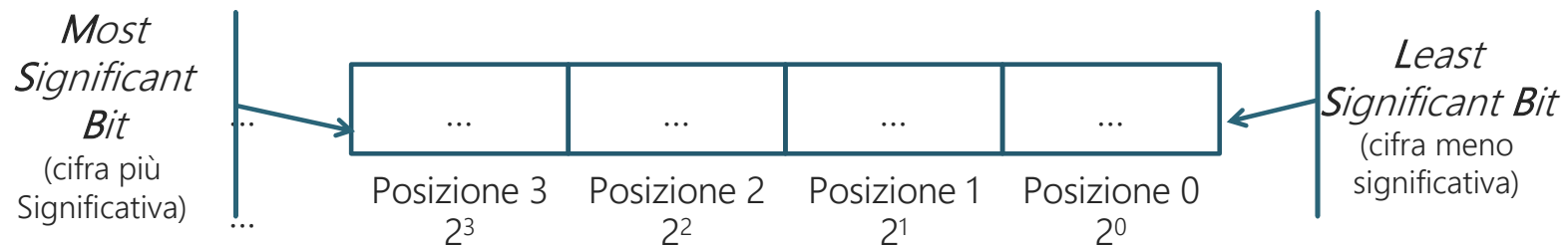
In un sistema **posizionale** a posizioni uguali corrispondono 'pesi' uguali delle cifre. Quindi utilizzando un sistema posizionale alcune operazioni di calcolo sono molto semplici grazie alla disposizione in colonna di cifre dello stesso peso. Provate a svolgere l'addizione utilizzando un sistema non posizionale, per esempio con i numeri romani...

Rappresentazione dei numeri interi

È possibile memorizzare l'informazione utilizzando solo i simboli 0 e 1?

Sistema di numerazione posizionale in base 2

- Alfabeto di simboli (cifre binarie) {0,1}
- Peso di ciascun simbolo dipendente dalla posizione



Ho 2 simboli e il 'peso' associato a ciascuna posizione è dato dalle potenze di 2: 2^0 per la prima posizione, 2^1 per la seconda, 2^2 per la terza e così via

(ecco da dove viene la tabella di corrispondenza vista prima)

Rappresentazione dei numeri interi

Se chiamiamo a_i la cifra in i -esima posizione, l'insieme di simboli $(a_3a_2a_1a_0)_2$ rappresenta il numero

a_3	a_2	a_1	a_0
-------	-------	-------	-------

$$= a_3 \cdot 2^3 + a_2 \cdot 2^2 + a_1 \cdot 2^1 + a_0 \cdot 2^0 =$$
$$= a_3 \cdot 8 + a_2 \cdot 4 + a_1 \cdot 2 + a_0 \cdot 1$$

Esempio di *cambiamento di base*

Consideriamo l'insieme di simboli $(\boxed{1}\boxed{1}\boxed{0}\boxed{0}\boxed{1})_2$

$$\underline{1} \cdot 2^4 + \underline{1} \cdot 2^3 + \underline{0} \cdot 2^2 + \underline{0} \cdot 2^1 + \underline{1} \cdot 2^0 =$$
$$= 1 \cdot 16 + 1 \cdot 8 + 0 \cdot 4 + 0 \cdot 2 + 1 \cdot 1 = 25$$
$$(11001)_2 = (25)_{10}$$

Rappresentazione dei numeri interi

Sistema di numerazione posizionale

Base 10

- Alfabeto di simboli $\{0,1,2,3,4,5,6,7,8,9\}$
- Peso associato alla posizione:
... $10^4, 10^3, 10^2, 10^1, 10^0$

Base 2

- Alfabeto di simboli $\{0,1\}$
- Peso associato alla posizione:
... $2^4, 2^3, 2^2, 2^1, 2^0$

- Abbiamo espresso i numeri *interi positivi* nella loro rappresentazione binaria naturale
- La cifra che può assumere solo due valori, **0** o **1**, è chiamata **bit** (binary digit)

Rappresentazione dei numeri interi

Considerazioni

- Abbiamo esaminato la conversione da **base 2 a base 10**.
- Non abbiamo visto la conversione da base 10 a base 2 (ma è possibile).
- Nel sistema binario si possono rappresentare anche numeri **negativi** o con **parte decimale**; in generale però la rappresentazione dei reali^(*) è **approssimata**. Esempio in Python:
`1 - 1/3 - 1/3 - 1/3 → 1.1102230246251565e-16` (*invece di 0*)
- In base 2 si possono eseguire le **usuali operazioni aritmetiche**.

(*) La rappresentazione dei reali non rientra tra gli obiettivi del corso: l'importante è ricordare che è **approssimata** e quindi può introdurre errori. Per approfondire: "rappresentazione in virgola mobile", oppure D. Goldberg, [*What Every Computer Scientist Should Know About Floating-Point Arithmetic*](#).

Rappresentazione dei numeri interi

RIASSUMENDO:

- I sistemi di elaborazione presi in esame sono costituiti da elementi che possono assumere i valori **0** o **1**
- Ciascuno di questi elementi può memorizzare **un bit**, che può essere utilizzato nella rappresentazione degli interi.

Non vedremo nel dettaglio come il computer può rappresentare altri enti numerici (numeri negativi, frazionari, ecc.)

Il byte e i suoi multipli

Un **byte** è un insieme di **8 bit**. Con un byte è possibile rappresentare $2^8 = 256$ possibili valori distinti.

I multipli del byte, riportati in tabella, sono definiti per mezzo delle potenze di 2. Tuttavia negli usi pratici (non specialistici) si possono approssimare i valori esatti agli usuali valori dei multipli delle unità di misura.

Nome	Simbolo	Valore esatto	Valore approssimato
kilobyte	KB	$2^{10} = 1024$ byte	$10^3 = 1000$ byte
megabyte	MB	$2^{20} = 1024 * 1024$ byte = 1'048'576 byte	$10^6 = 1$ milione di byte
gigabyte	GB	$2^{30} = 1024 * 1024 * 1024$ byte	$10^9 = 1$ miliardo di byte
terabyte	TB	$2^{40} = 1024 * 1024 * 1024 * 1024$ byte	$10^{12} = 1000$ miliardi di byte
...			

Il byte e i suoi multipli

Esempio:

12 500 byte equivalgono a *circa* 12.5 KB

25 700 000 byte equivalgono a *circa* 25.7 MB

32 200 MB equivalgono a *circa* 32.2 GB

(NB il separatore decimale utilizzato è il *punto*, non la *virgola*, secondo la notazione anglosassone)

Nome	Simbolo	Valore esatto	Valore approssimato
kilobyte	KB	$2^{10} = 1024$ byte	$10^3 = 1000$ byte
megabyte	MB	$2^{20} = 1024 * 1024$ byte = 1'048'576 byte	$10^6 = 1$ milione di byte
gigabyte	GB	$2^{30} = 1024 * 1024 * 1024$ byte	$10^9 = 1$ miliardo di byte
terabyte	TB	$2^{40} = 1024 * 1024 * 1024 * 1024$ byte	$10^{12} = 1000$ miliardi di byte
...			

Somma utilizzando il sistema binario

Esempio di somma: partiamo dalle **cifre meno significative** (a destra)

Caso 1: $0 + 0 = 0$

$$\begin{array}{r} 1 \quad 1 \quad 0 \quad + \\ 0 \quad 1 \quad 0 \quad = \\ \hline \end{array}$$

The diagram shows a binary addition of 110 and 010. The least significant bit (rightmost) is highlighted in a blue box. Above the 0 in the first row and the 0 in the second row is a red '0', indicating no carry. Below the horizontal line, under the highlighted bit, is a red '0', indicating the result of the addition.

La somma delle **cifre meno significative** non genera alcun riporto.

Caso 2: $0 + 1 = 1$

$$\begin{array}{r} 1 \quad 1 \quad 0 \quad + \\ 0 \quad 1 \quad 1 \quad = \\ \hline \end{array}$$

The diagram shows a binary addition of 110 and 011. The least significant bit (rightmost) is highlighted in a blue box. Above the 0 in the first row and the 1 in the second row is a red '0', indicating no carry. Below the horizontal line, under the highlighted bit, is a red '1', indicating the result of the addition.

Anche in questo caso la somma non genera alcun riporto.

Caso 3: $1 + 1 = 10$

$$\begin{array}{r} 1 \quad 1 \quad 1 \quad + \\ 0 \quad 1 \quad 1 \quad = \\ \hline \end{array}$$

The diagram shows a binary addition of 111 and 011. The least significant bit (rightmost) is highlighted in a blue box. Above the 1 in the first row and the 1 in the second row is a red '1', indicating a carry. Below the horizontal line, under the highlighted bit, is a red '0', indicating the result of the addition.

Il risultato non si può scrivere con una sola cifra: scriviamo 0 nella somma parziale e generiamo un riporto di 1.

Somma utilizzando il sistema binario

Esempio di somma

Caso 1: $0 + 0 = 0$

1	0		
1	1	0	+
0	1	0	=

1	0	0	0
---	---	---	---

$0 + 0 = 0 \rightarrow 0$, nessun riporto
 $1 + 1 = 10 \rightarrow 0$, riporto 1
 $1 + 0 + 1 = 10 \rightarrow 0$, riporto 1
L'ultimo riporto diventa la prima cifra: 1000

Caso 2: $0 + 1 = 1$

1	0		
1	1	0	+
0	1	1	=

1	0	0	1
---	---	---	---

$0 + 1 = 1 \rightarrow 1$, nessun riporto
 $1 + 1 = 10 \rightarrow 0$, riporto 1
 $1 + 0 + 1 = 10 \rightarrow 0$, riporto 1
L'ultimo riporto diventa la prima cifra: 1001

Caso 3: $1 + 1 = 10$

1	1		
1	1	1	+
0	1	1	=

1	0	1	0
---	---	---	---

$1 + 1 = 10 \rightarrow 0$, riporto 1
 $1 + 1 + 1 = 11 \rightarrow 1$, riporto 1
 $1 + 0 + 1 = 10 \rightarrow 0$, riporto 1
L'ultimo riporto diventa la prima cifra: 1010

ESERCIZI

1. Convertite in base 10 una serie di numeri in base 2.
2. Data una coppia di numeri in base 2:
 - calcolate la somma (restando in base 2)
 - convertite i due numeri e la somma in base 10
 - verificate che la somma sia corretta

Esempio svolto

Somma in base 2

$$\begin{array}{r} 0101 + \\ 0100 = \\ \hline 1001 \end{array}$$

Verifica in base 10

$$0101 \rightarrow 5$$

$$0100 \rightarrow 4$$

$$1001 \rightarrow 9$$

$5 + 4 = 9$: la somma è corretta

Rappresentazione dei caratteri

- Il **codice ASCII esteso** (*) utilizza 8 bit: può quindi rappresentare fino a **256 caratteri**.
- È uno 'standard de facto'
- Con un byte (8 bit) si rappresenta un carattere
- La sequenza di tasti **ALT + codice ASCII** richiama il carattere specificato (ad es. ALT+0224 → à)

Oltre ai caratteri dell'ASCII a 7 bit, l'ASCII esteso rappresenta:

lettere accentate di vari alfabeti

à è é ñ ü ç

simboli monetari e matematici

£ ¥ ¢ ± ÷ ½

caratteri grafici

■ ■ ▣ ▤ ▥

lettere dell'alfabeto greco

α β π Σ Ω

(*) esistono versioni leggermente differenti dell'ASCII esteso. Citiamo ad esempio la versione 'Latin-1', più formalmente ISO/IEC 8859-1, riportata in http://en.wikipedia.org/wiki/ISO_8859-1

Rappresentazione dei caratteri

- Anche 256 caratteri sono insufficienti: mancano i caratteri degli alfabeti cirillico, arabo, ecc.
- **UNICODE** (*) è un sistema di codifica ideato per risolvere questo e altri problemi
- UNICODE si basa su una codifica a 8, 16 o 32 bit e assegna un numero univoco a ogni carattere,
 - indipendentemente dalla piattaforma
 - indipendentemente dall'applicazione
 - indipendentemente dalla lingua

Nota: la codifica più diffusa, UTF-8, usa da 1 a 4 byte per carattere e rappresenta tutti i caratteri UNICODE.

(*) <http://www.unicode.org/standard/translations/italian.html>

Quanti caratteri si possono distinguere con:

8 bit $\rightarrow 2^8 =$

256 caratteri

16 bit $\rightarrow 2^{16} =$

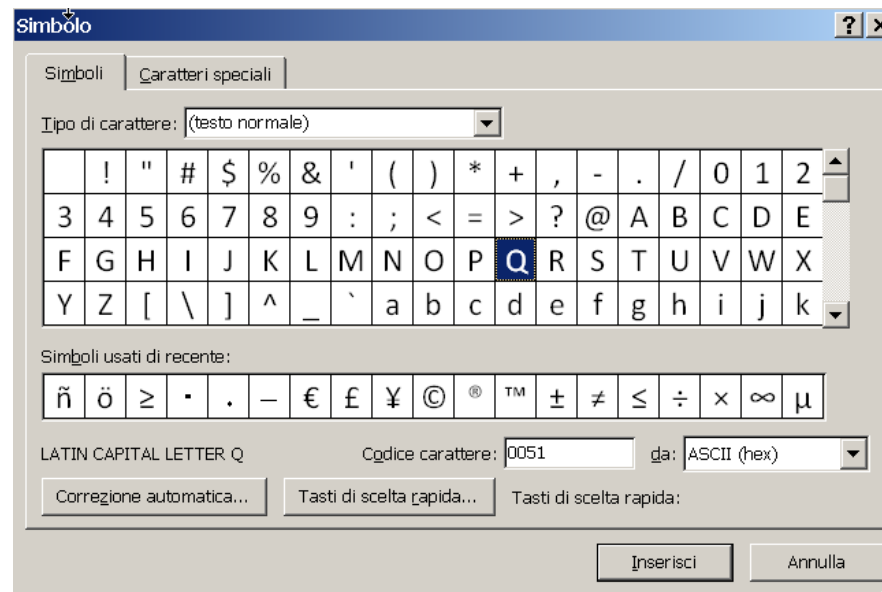
65.536 caratteri

32 bit $\rightarrow 2^{32} =$

4,29 miliardi caratteri

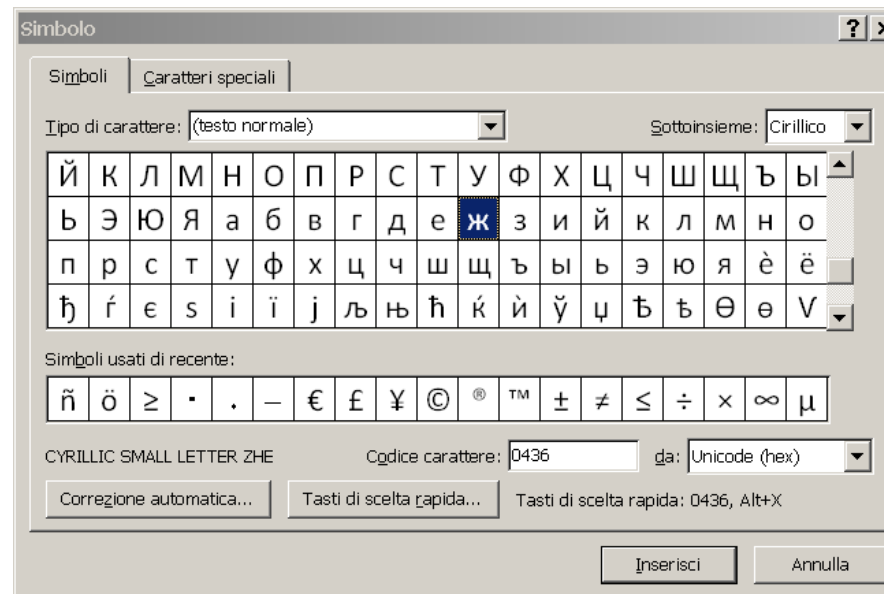
Rappresentazione dei caratteri

➤ Esempio inserimento caratteri ASCII (Windows)



Rappresentazione dei caratteri

- Esempio inserimento caratteri UNICODE (Windows)

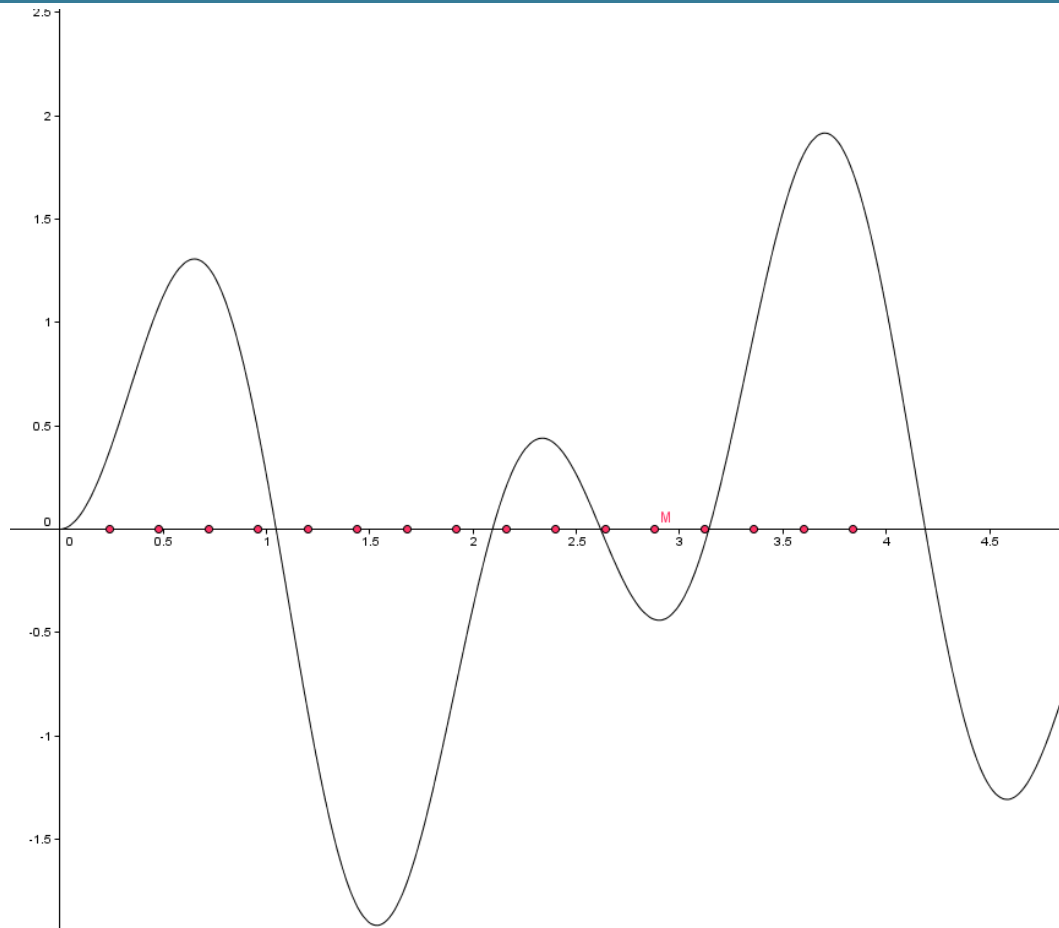


Rappresentazione delle informazioni

Problema: trasformare un segnale da **analogico** a **digitale**

- Segnale **analogico**: rappresentazione di una grandezza mediante una grandezza *analogica*. È un segnale continuo che può assumere una gamma infinita di valori
- Segnale **digitale**: rappresentazione di una grandezza mediante un insieme di *valori numerici*.

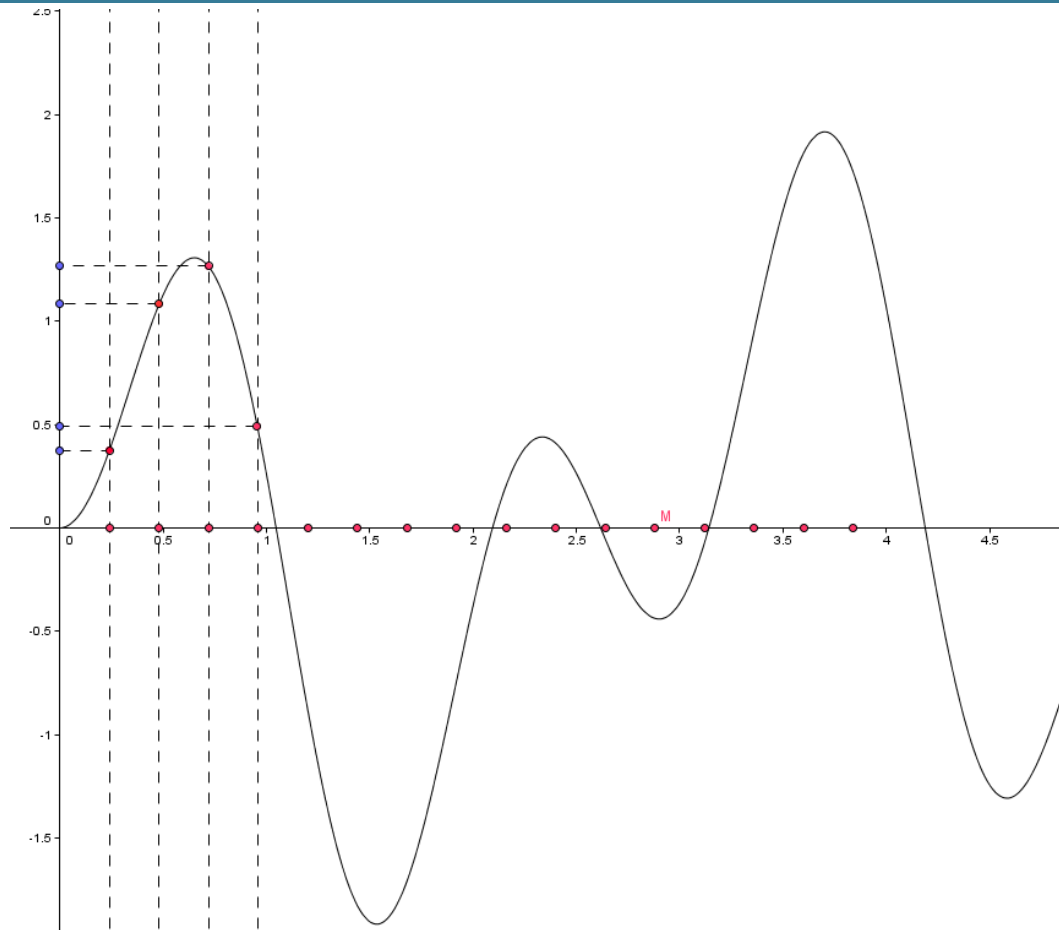
Campionamento e quantizzazione



Esempio di grandezza analogica

Si effettua un campionamento ad intervalli di tempo prefissati

Campionamento e quantizzazione



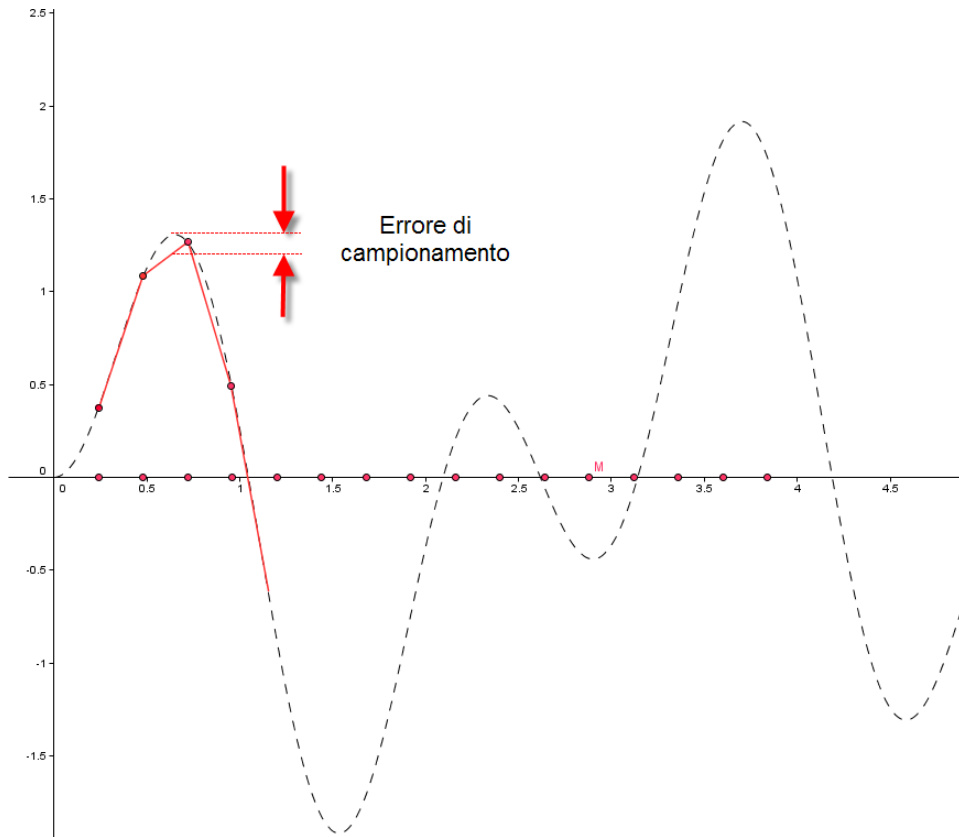
Otteniamo una serie di misure che rappresentano il valore della grandezza in esame negli istanti di tempo considerati

Campionamento e quantizzazione

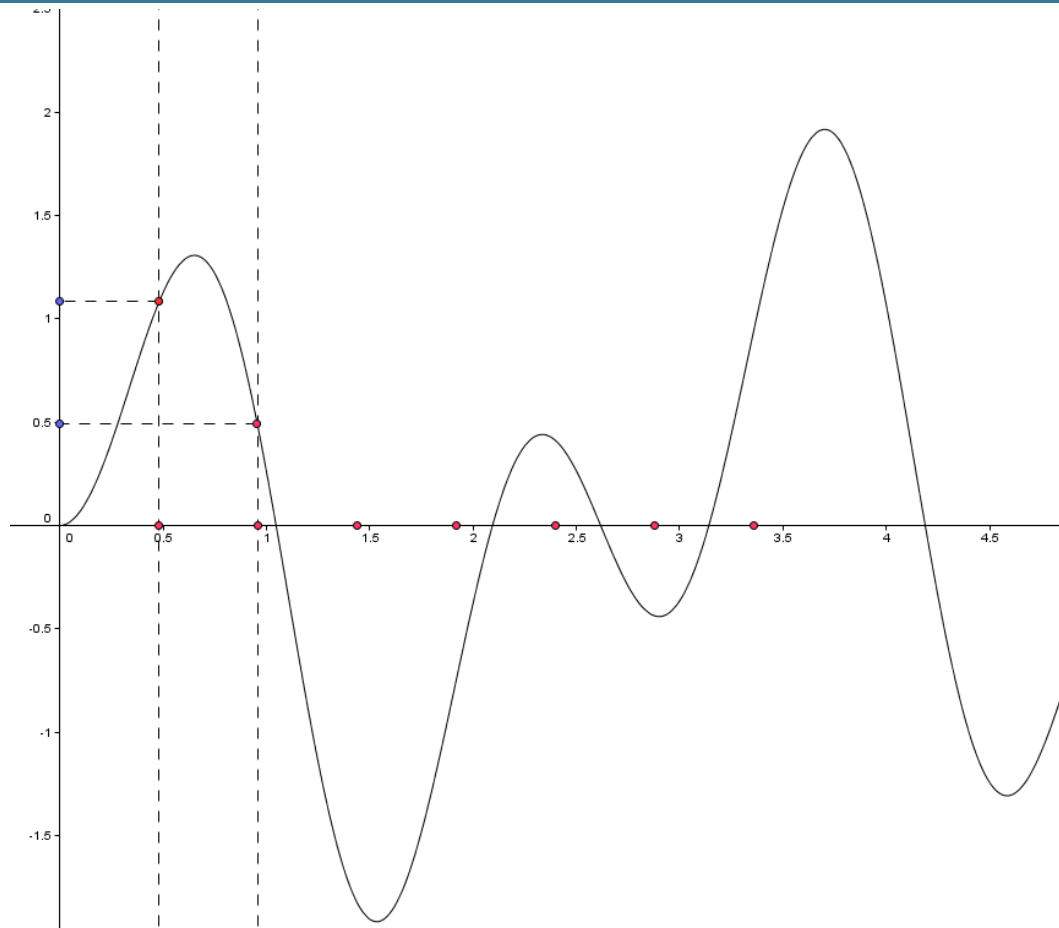
L'obiettivo è ricostruire l'informazione originale mediante le misure ottenute.

Anche ipotizzando di ottenere *misure perfette* si commette un errore dovuto al fatto che il segnale analogico è costituito da un insieme infinito di valori, mentre il processo di campionamento considera un insieme finito di misure.

La spezzata in rosso rappresenta il tentativo di ricostruire il segnale originale a partire dalle misure. Si nota che nell'istante della misura la differenza tra segnale vero e segnale ricostruito è nulla, mentre negli altri punti commettiamo un errore.

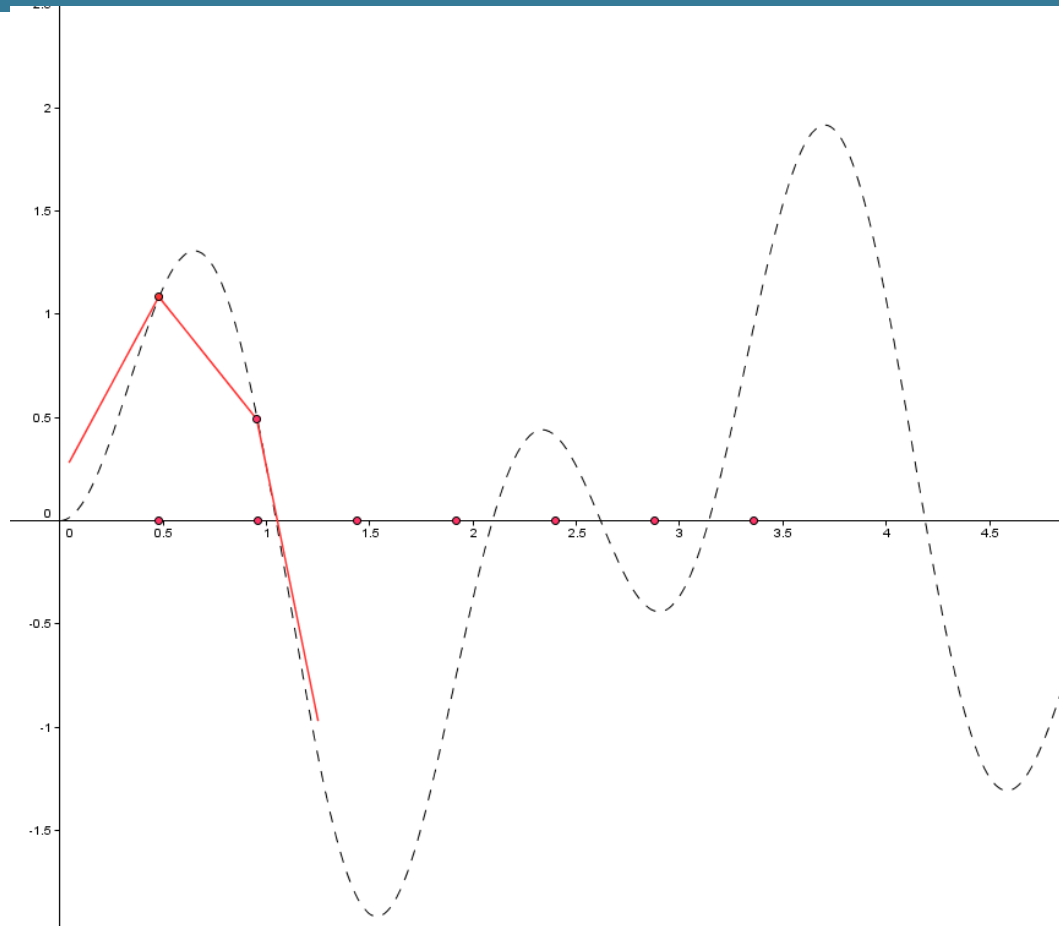


Campionamento e quantizzazione



Se diradiamo ulteriormente
gli atti di campionamento...

Campionamento e quantizzazione

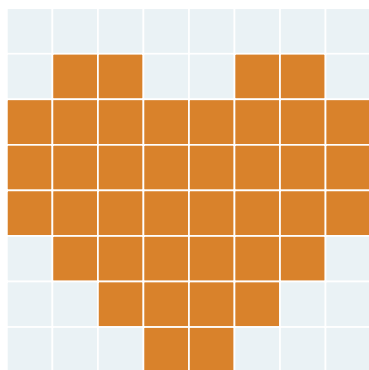


... l'errore aumenta.

L'errore di campionamento è legato alla **frequenza di campionamento** e alla **frequenza del segnale**. Intuitivamente se il segnale analogico varia con rapidità (frequenze elevate) è necessario aumentare il numero di campioni per ricostruire il segnale. Aumentare il numero di campioni significa aumentare la frequenza di campionamento (o diminuire l'intervallo tra un campione e il successivo)

Codifica Immagini (cenni)

La **grafica bitmap**, o **grafica raster** (griglia), è una tecnica^(*) utilizzata per rappresentare un'immagine. Un'immagine descritta con questo tipo di grafica è chiamata *immagine bitmap* o *immagine raster*.



Ogni tessera è un **pixel**
(qui: immagine 8 × 8 a 2 colori)

L'immagine viene vista come un *mosaico*: a ogni tessera, chiamata **pixel** (*picture element*), viene associato uno specifico colore.

Il colore può essere definito con due tecniche:

Pochi colori (max 256)

Si crea un elenco (*palette*) dei colori usati; per ogni pixel si memorizza l'indice del suo colore nell'elenco.

Molti colori

A ogni pixel si associa un valore numerico che definisce direttamente il colore.

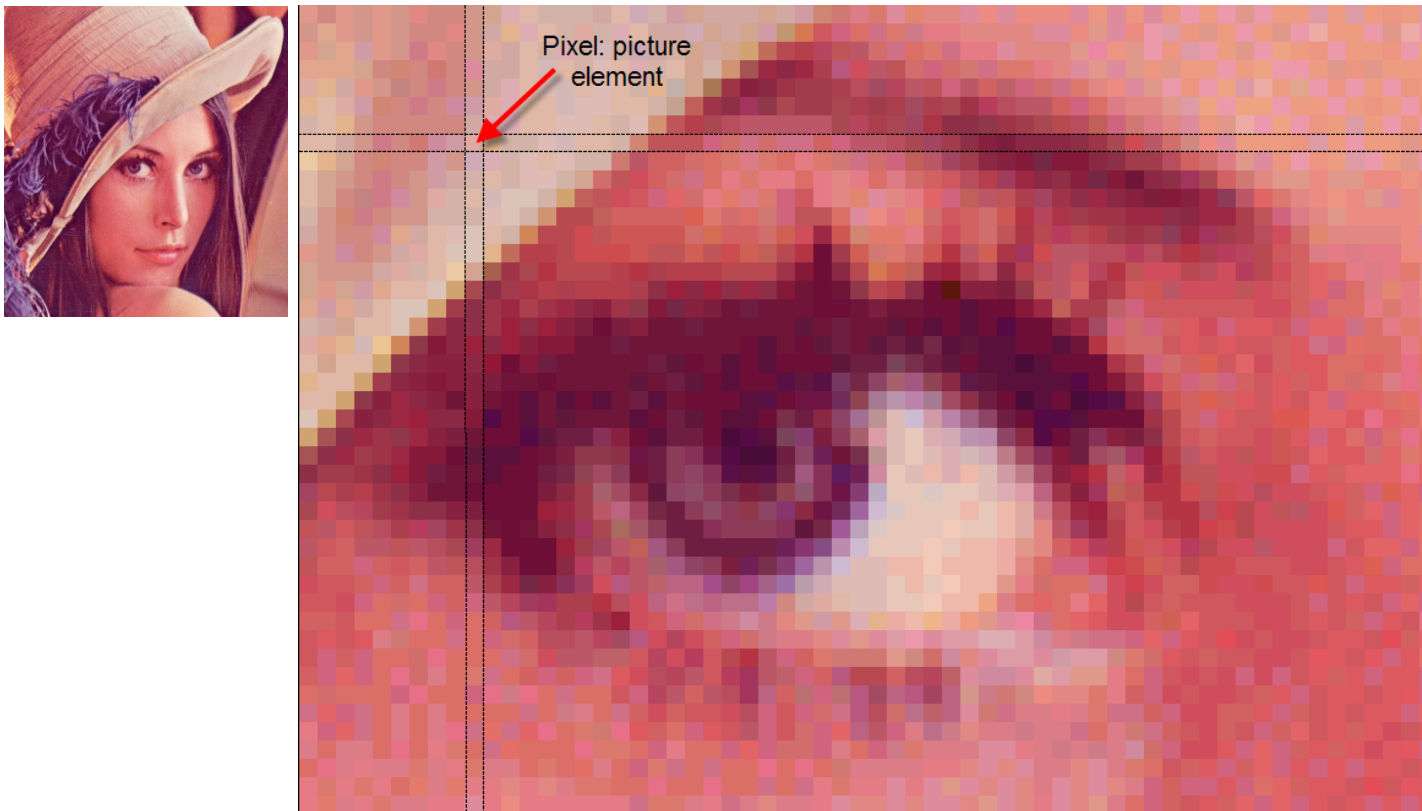
(*) Questo non è l'unico modo per rappresentare un'immagine. Nella *grafica vettoriale* un'immagine è descritta mediante un insieme di primitive geometriche che descrivono punti, linee, curve e poligoni che compongono l'immagine.

Codifica Immagini (cenni)

L'immagine rappresentata in questo modo è caratterizzata da due proprietà

- Risoluzione: è determinata dal numero di pixel contenuti nell'unità di misura considerata
- Profondità di colore: è definita dal numero di bit dedicati ad ogni pixel per descrivere il colore. Maggiore è il numero di bit, maggiore è il numero di colori che è possibile descrivere.

Codifica Immagini (cenni)



Compressione lossy

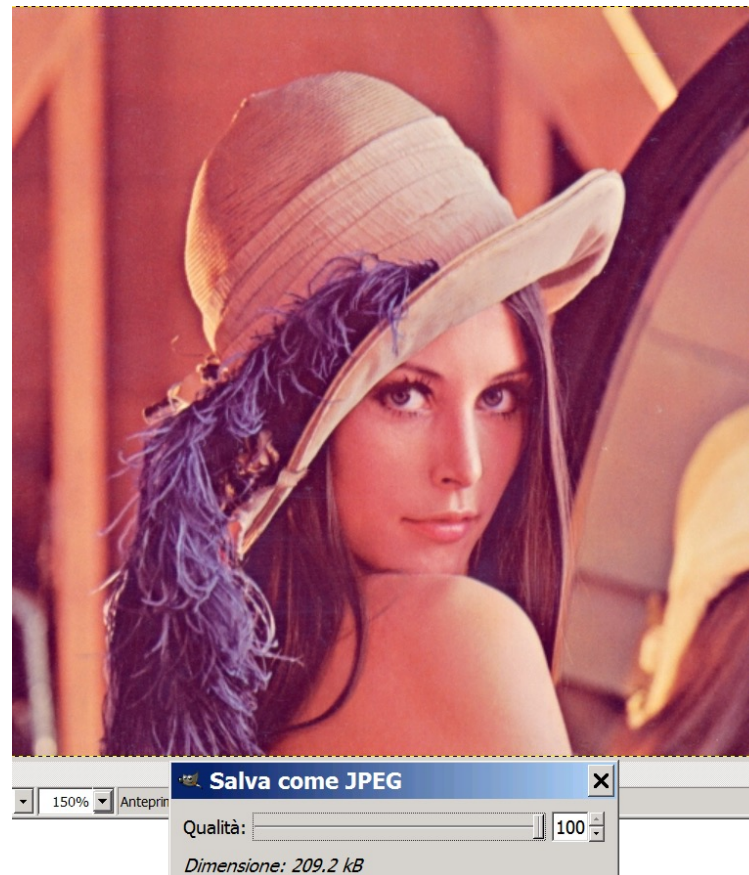


Lena Sjööblom (31 marzo 1951) ex modella svedese, playmate del mese di novembre 1972 della rivista Playboy.

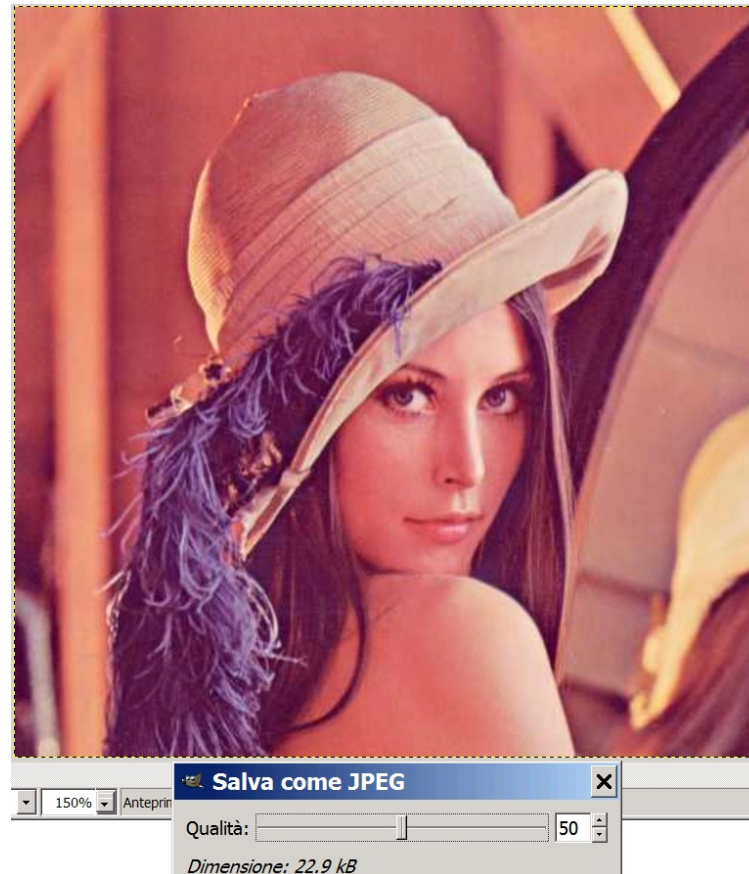


Lena 512x512, 262144 pixel,
RGB, 8 bit per colore
Dimensioni: $512 \times 512 \times 3 = 768$
KByte

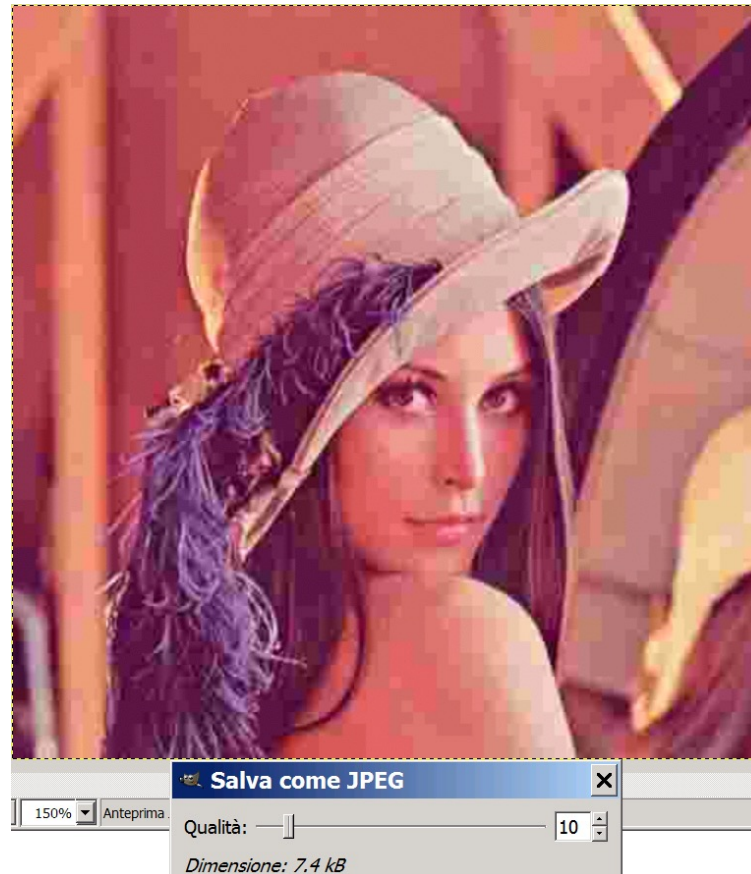
Compressione lossy



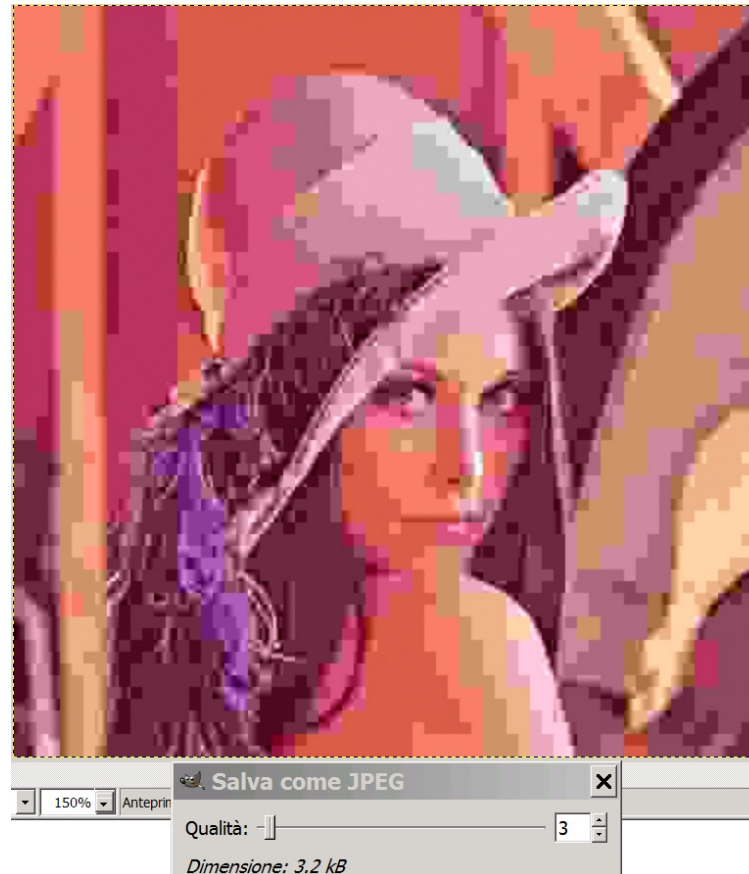
Compressione lossy



Compressione lossy



Compressione lossy



Il sistema di elaborazione

Cosa serve per elaborare informazioni in modo automatico?

In questa sezione

1

Principi di funzionamento

Come si elaborano i bit

2

Algebra di Boole e porte logiche

Operazioni su 0 e 1 e circuiti che le realizzano

3

Il calcolatore moderno

Dagli automi del Settecento al computer

Principi di funzionamento

Come può un dispositivo elettronico svolgere operazioni aritmetiche o logiche?

Operazioni aritmetiche

Calcolano un numero

- somma
- moltiplicazione
- ...

Operazioni logiche

Rispondono a una domanda

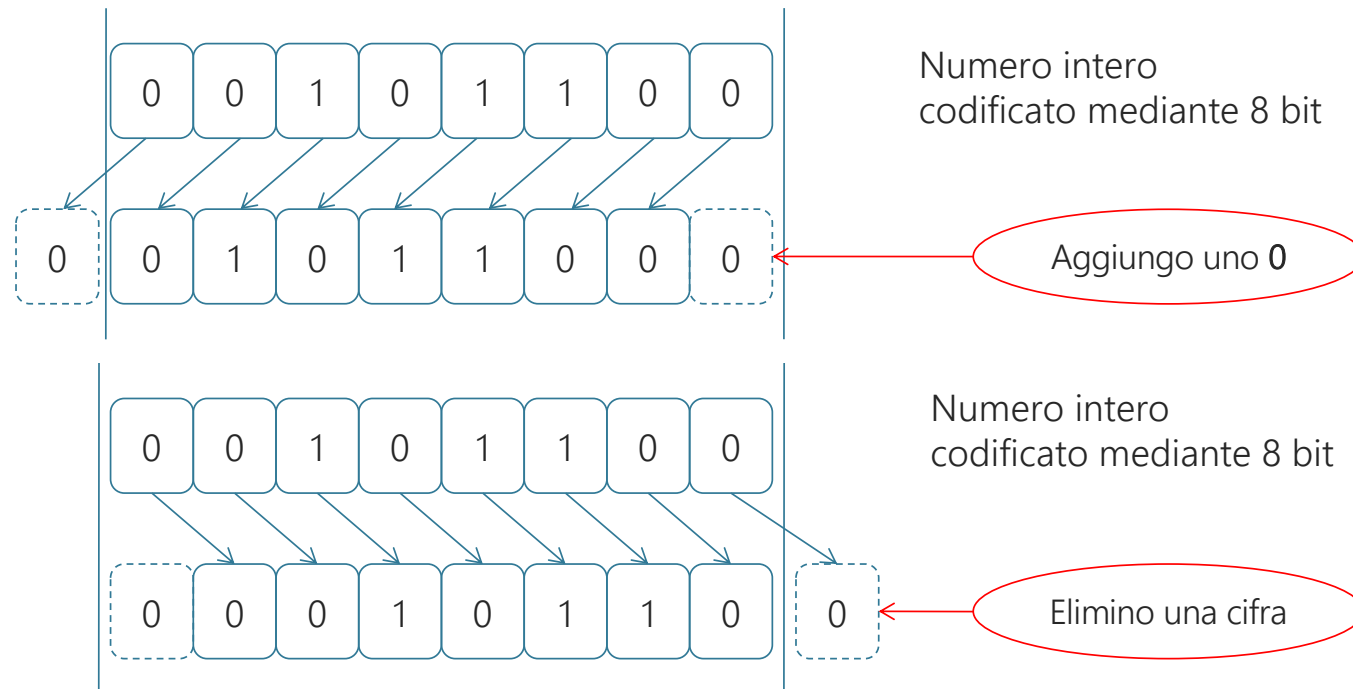
- Due numeri A e B sono uguali?
- Il numero A è maggiore di B?
- La parola A è uguale alla parola B?
- La parola A precede, nell'ordine alfabetico, la parola B?

La risposta a una domanda logica è **vero** o **falso**: per rappresentarla basta un bit.

Esempio di operazioni

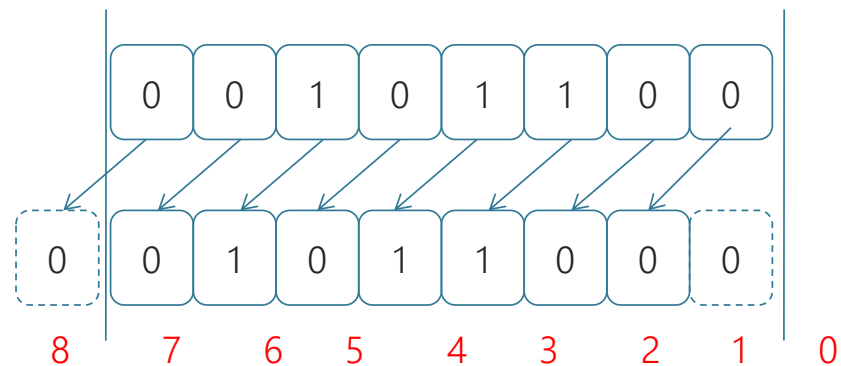
Esempio di realizzazione di operazione aritmetica

Cosa succede traslando tutti i bit a sinistra o a destra?



Esempio di operazioni

Numero intero
codificato mediante 8 bit

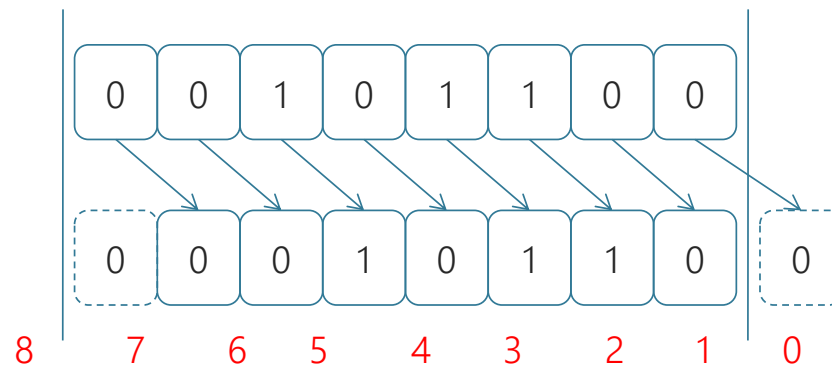


Analogia:
In base 10, aggiungere N
zeri significa moltiplicare per
 10^N

$$\begin{aligned} &0 \cdot 2^7 + 0 \cdot 2^6 + 1 \cdot 2^5 + 0 \cdot 2^4 + 1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 0 \cdot 2^0 \\ &0 \cdot 2^8 + 0 \cdot 2^7 + 1 \cdot 2^6 + 0 \cdot 2^5 + 1 \cdot 2^4 + 1 \cdot 2^3 + 0 \cdot 2^2 + 0 \cdot 2^1 \\ &2^1(0 \cdot 2^7 + 0 \cdot 2^6 + 1 \cdot 2^5 + 0 \cdot 2^4 + 1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 0 \cdot 2^0) \end{aligned}$$

Esempio di operazioni

Numero intero
codificato mediante 8 bit



Analogia:
In base 10, spostare la
virgola di n posti a sinistra
significa dividere per 10^N

$$\begin{aligned}
 &0 \cdot 2^7 + 0 \cdot 2^6 + 1 \cdot 2^5 + 0 \cdot 2^4 + 1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1 + 0 \cdot 2^0 \\
 &0 \cdot 2^6 + 0 \cdot 2^5 + 1 \cdot 2^4 + 0 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0 + 0 \cdot 2^0 \\
 &(0 \cdot 2^7 + 0 \cdot 2^6 + 1 \cdot 2^5 + 0 \cdot 2^4 + 1 \cdot 2^3 + 1 \cdot 2^2 + 0 \cdot 2^1) / 2^1 + 0 \cdot 2^0
 \end{aligned}$$

Dalle operazioni sui bit ai circuiti

- In base all'esempio precedente è facile immaginare come si possa realizzare un meccanismo (nel nostro caso, un circuito elettronico) che traslando i bit a destra o a sinistra effettui delle divisioni o moltiplicazioni per potenze di 2.
- All'interno della RAM (la memoria di lavoro del calcolatore) i bit sono memorizzati mediante cariche elettriche. È quindi semplice, almeno dal punto di vista 'di principio', immaginare un semplice collegamento che sposti una carica elettrica da una cella alla cella adiacente.

Algebra booleana

Come si calcola con due soli valori, 0 e 1?

In questa sezione

1

Definizione

L'insieme $B = \{0, 1\}$ e le operazioni definite su di esso

2

Operatori AND, OR, NOT, XOR

Come funzionano e le loro tabelle di verità

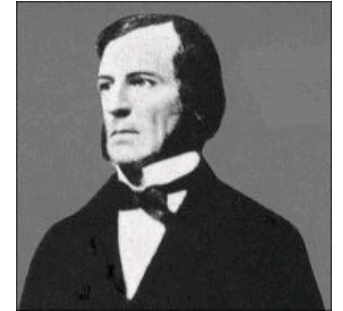


George Boole

George Boole (Lincoln, 2 novembre 1815 – Ballintemple, 8 dicembre 1864)

Matematico e logico britannico

È considerato il fondatore della logica matematica.



Nel 1854 pubblicò 'Le leggi del pensiero', nel quale tentò di ricondurre le composizioni degli enunciati a semplici operazioni algebriche.

George Boole, *An Investigation of the Laws of Thought, on Which are founded the Mathematical Theories of Logic and Probabilities*, Dover Publications. Reprinted 1858 and 1973

Per un percorso storico:

Il calcolatore universale. Da Leibniz a Turing [Martin Davis, 2012]

Algebra Booleana

È un'algebra che ha come supporto un insieme **B** formato da due soli elementi:

$$\mathbf{B} = \{0, 1\}$$

AND

$$\mathbf{B} \times \mathbf{B} \rightarrow \mathbf{B}$$

Operazione binaria: combina due valori

OR

$$\mathbf{B} \times \mathbf{B} \rightarrow \mathbf{B}$$

Operazione binaria: combina due valori

NOT

$$\mathbf{B} \rightarrow \mathbf{B}$$

Operazione unaria: agisce su un solo valore

Le porte logiche realizzano fisicamente le operazioni dell'algebra di Boole.

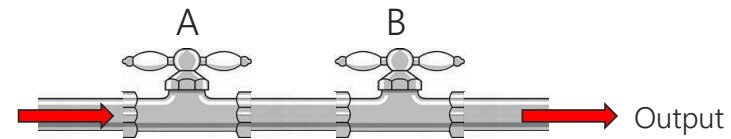
Vediamo come è possibile, almeno da un punto di vista 'di principio', realizzarle.

Porte Logiche

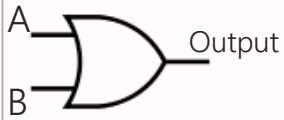
AND



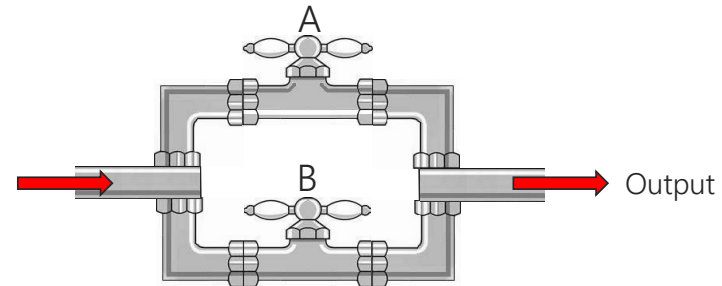
INPUT		OUTPUT
A	B	A AND B
0	0	0
0	1	0
1	0	0
1	1	1



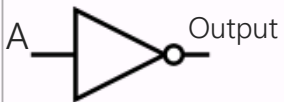
OR



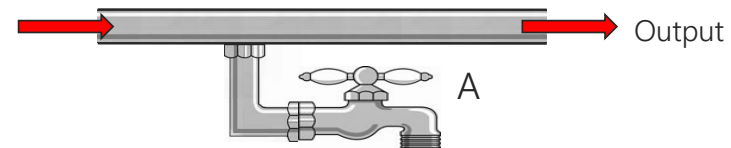
INPUT		OUTPUT
A	B	A OR B
0	0	0
0	1	1
1	0	1
1	1	1



NOT



INPUT	OUTPUT
A	NOT A
0	1
1	0



Porte Logiche

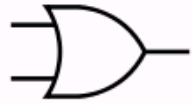
AND



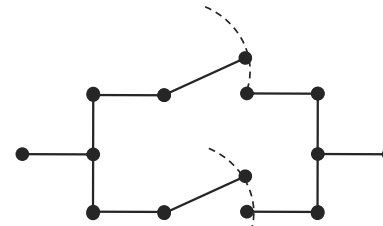
INPUT		OUTPUT
A	B	A AND B
0	0	0
0	1	0
1	0	0
1	1	1



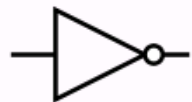
OR



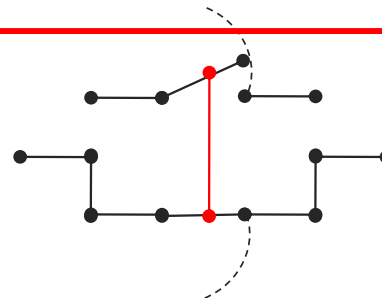
INPUT		OUTPUT
A	B	A OR B
0	0	0
0	1	1
1	0	1
1	1	1



NOT



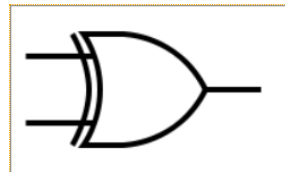
INPUT	OUTPUT
A	NOT A
0	1
1	0



eXclusive OR

XOR (eXclusive OR) è una porta logica che riceve in ingresso "n" valori e restituisce "1" in uscita se e solo se il numero di ingressi che presentano il valore logico "1" è dispari.

➤ Nell'esempio, una porta XOR a 2 ingressi



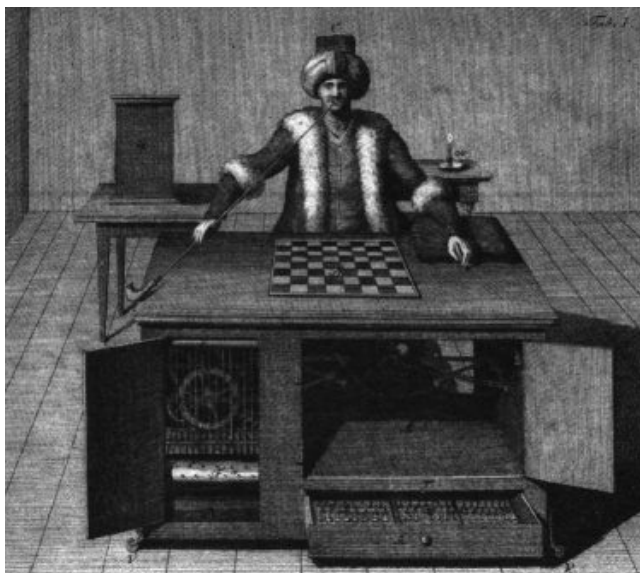
INPUT		OUTPUT
A	B	$A \oplus B$
0	0	0
0	1	1
1	0	1
1	1	0

Il calcolatore moderno

Dagli automi del Settecento al computer

Le origini – il turco

- 1770 - Wolfgang von Kempelen costruisce un automa in grado di giocare a scacchi e lo esibisce alla corte dell'Imperatrice Maria Teresa d'Austria



(in realtà si trattava di un inganno; il manichino era manovrato da un essere umano nascosto all'interno)

Il primo calcolatore moderno

- **ENIAC** (Electronic Numerical Integrator And Computer)
- Fu progettato presso l'Università di Pennsylvania e completato nel **1946**.
- L'attività dell'ENIAC fu finanziata inizialmente dalla Marina degli Stati Uniti, interessata al calcolo delle traiettorie balistiche

John von Neumann



János Lajos Margittai Neumann

naturalizzato statunitense come John von Neumann

Budapest, 28 dicembre 1903 – Washington, 8 febbraio 1957

È universalmente noto come il padre dell'omonima **architettura dei calcolatori**, impiegata nella maggior parte dei computer moderni.

Fanno eccezione, ad esempio, le architetture parallele e quelle basate su reti neurali.

Architettura: insieme delle parti di un calcolatore e delle loro interconnessioni.

John von Neumann

L'uomo: qualche aneddoto

"Quando vedeva la madre assorta le chiedeva che cosa stesse calcolando; in bagno si portava due libri, per paura di finire di leggerne uno prima di aver terminato [...]"

"Quando si dichiarò alla donna che poi sposò [le disse] 'lo e te potremmo divertirci insieme, visto che ad entrambi piace bere'."

Fonte: <http://www.vialattea.net/odifreddi/bio/neumann.htm>



sAifer Lab
Joint lab on Safety and Security of AI

www.saiferlab.ai

John von Neumann

Il contributo ai progetti militari

"[...] Il suo risultato più famoso nel campo [degli armamenti] fu la scoperta che le bombe di grandi dimensioni sono più devastanti se scoppiano prima di toccare il suolo [...] L'applicazione più infame del risultato si ebbe [...] quando le più potenti bombe della storia detonarono sopra il suolo di Hiroshima e Nagasaki, all'altezza calcolata da von Neumann affinché esse producessero il maggior danno aggiuntivo."

Fonte: <http://www.vialattea.net/odifreddi/bio/neumann.htm>

Alan Turing



Alan Mathison Turing

Londra, 23 giugno 1912 – Wilmslow, 7 giugno 1954

Considerato uno dei padri dell'informatica, introdusse la **macchina ideale** e il **test** che portano il suo nome.

Macchina di Turing

È un modello teorico di calcolo.

- Serve a capire che cosa può fare (e che cosa non può fare), in linea teorica, un sistema di elaborazione delle informazioni.
- Gli attuali calcolatori sono tutti Turing-equivalenti.

Test di Turing [3]

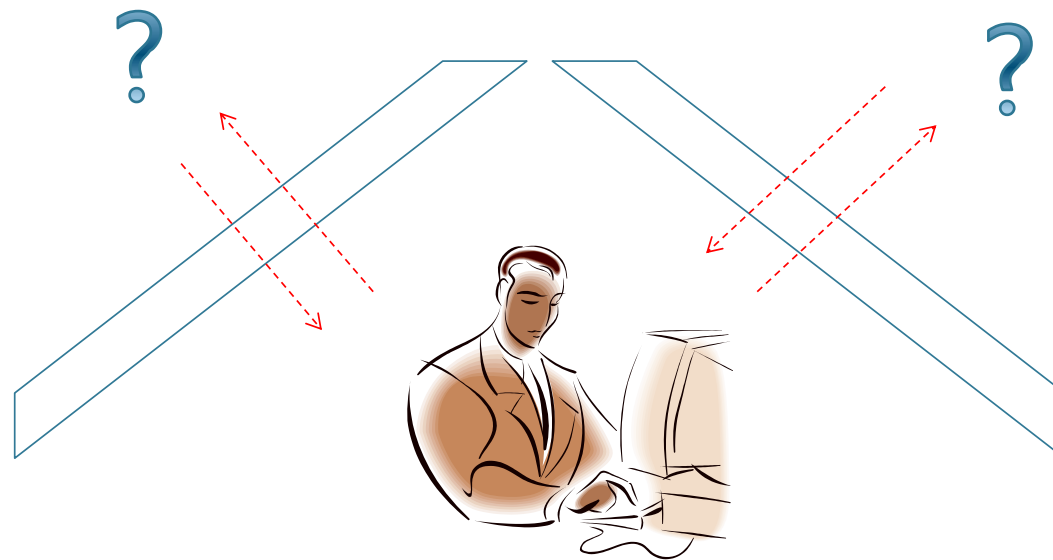
È un criterio per stabilire se una macchina mostri 'intelligenza'.

- Si basa sul confronto tra il comportamento di una macchina e quello di un essere umano.

Alan Turing

- 1952 ● Fu arrestato per omosessualità e sottoposto a un 'trattamento ormonale per la riduzione della libido', che lo rese impotente e causò lo sviluppo del seno.
- 1954 ● Morì suicida per avvelenamento da cianuro di potassio, probabilmente mangiando una mela avvelenata.
- 2013 ● Il governo britannico si scusò pubblicamente (2009) e la regina Elisabetta II gli concesse la grazia postuma (2013).

Test di Turing



Computing Machinery and Intelligence (Turing, 1950)

25

COMPUTING MACHINERY AND INTELLIGENCE

A. M. Turing

Source: *Mind* LIX(236), Oct. 1950: 433–60.

1 The imitation game

I propose to consider the question, 'Can machines think?' This should begin with definitions of the meaning of the terms 'machine' and 'think'. The definitions might be framed so as to reflect so far as possible the normal use of the words, but this attitude is dangerous. If the meaning of the words 'machine' and 'think' are to be found by examining how they are commonly used it is difficult to escape the conclusion that the meaning and the answer to the question, 'Can machines think?' is to be sought in a statistical survey such as a Gallup poll. But this is absurd. Instead of attempting such a definition I shall replace the question by another, which is closely related to it and is expressed in relatively unambiguous words.

"Computing machinery and intelligence",
1950, *Mind*

'Anti-Test di Turing'

CAPTCHA



Completely automated public Turing test to tell computers and humans apart
(Test di Turing pubblico e completamente automatico per distinguere computer e umani)

Test composto da una o più domande e risposte per determinare se l'utente sia un umano e non un computer

Macchina di von Neumann

La **macchina di von Neumann** (o **architettura di von Neumann**) è uno schema basato su quattro componenti fondamentali.

Architettura di un calcolatore: insieme delle parti e delle loro interconnessioni.

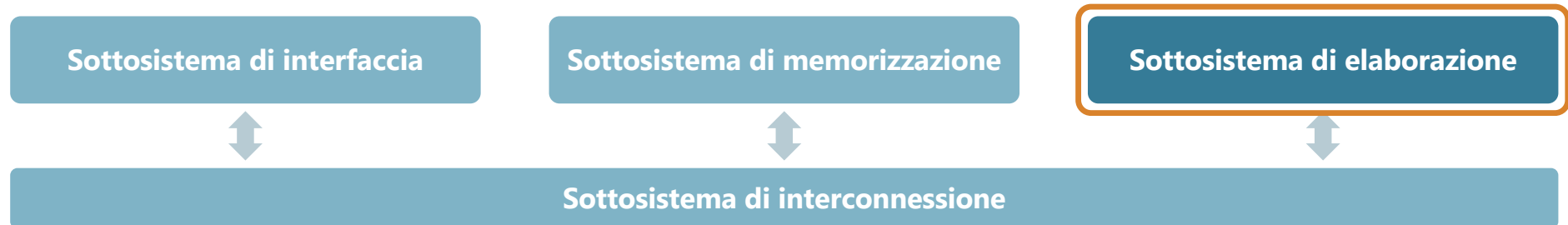


Macchina di von Neumann

Questa architettura si distingue per il concetto di *programma memorizzato*: i dati e le istruzioni che operano su di essi sono codificati e memorizzati sullo stesso dispositivo.



Sottosistema di elaborazione



Il sottosistema di elaborazione **esegue le istruzioni** e **controlla il flusso** di dati e istruzioni.

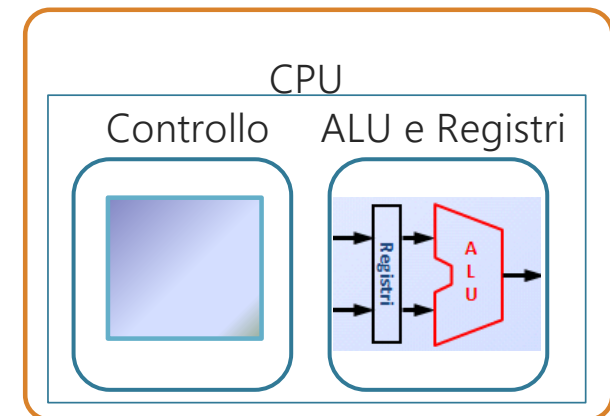
ALU

Unità aritmetico-logica

Esegue le operazioni aritmetiche e logiche.

Unità di controllo

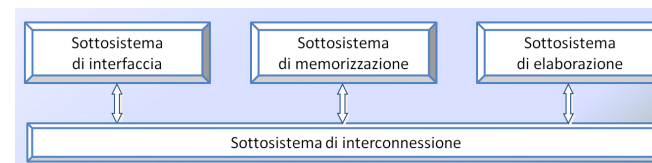
Stabilisce quale istruzione deve eseguire la ALU, su quali dati e dove depositare il risultato.



Macchina di von Neumann

Schema di funzionamento

- I programmi sono composti da istruzioni codificate in binario:
 - Istruzioni di elaborazione (ad es. operazioni numeriche)
 - istruzioni di trasferimento di dati tra due componenti della macchina
- Il funzionamento è un ciclo continuo: il sottosistema di elaborazione
 - estrae le istruzioni dalla memoria principale
 - le decodifica (determina l'operazione da eseguire e i dati su cui eseguirle)
 - le esegue

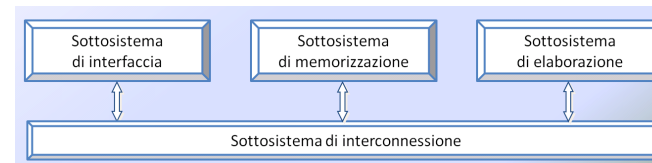


Sottosistema di memorizzazione

Memoria centrale (*memoria primaria dell'elaboratore*)

È chiamata anche memoria RAM (Random Access Memory)

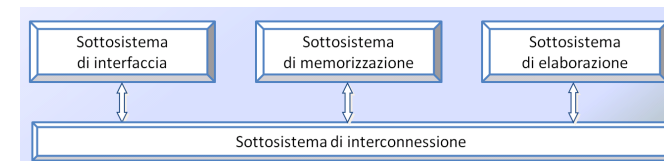
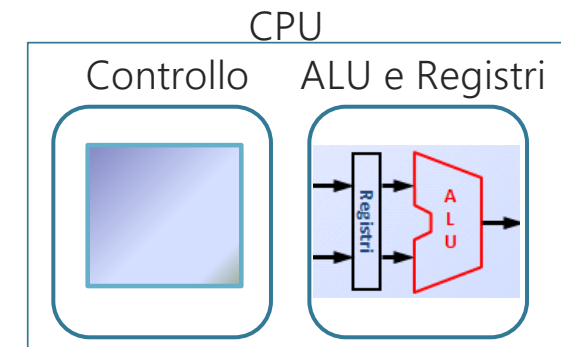
- Conserva (sotto forma di bit) i programmi in esecuzione ed i dati su cui essi operano
- È realizzata mediante dispositivi elettronici a due stati
- È una memoria volatile
- Velocità di accesso molto elevata



Sottosistema di elaborazione

Il **Processore** (CPU, Central Processing Unit) costituito da

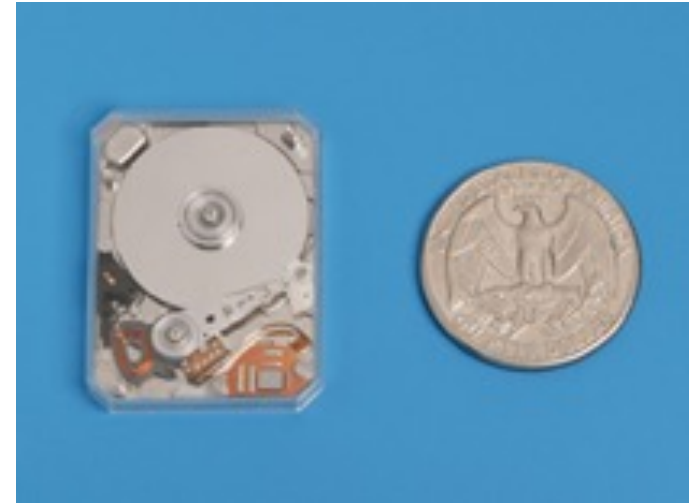
- Unità Aritmetico-Logica (ALU, Arithmetic and Logic Unit)
 - Modulo capace di eseguire un certo insieme di **operazioni aritmetiche e logiche**
- Unità di controllo:
 - recupera le istruzioni dalla memoria centrale
 - decodifica le istruzioni
 - trasferisce i dati necessari
 - controlla le unità coinvolte nell'esecuzione



Memorie di massa – dischi magnetici



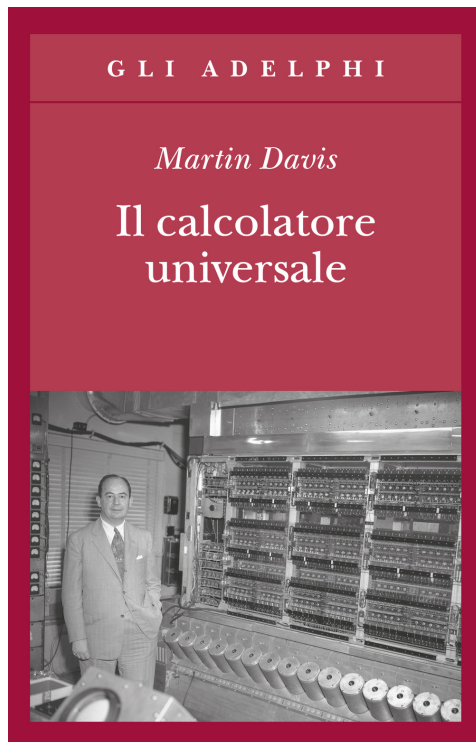
Il primo Hard Disk (settembre 1956)
Con la capienza di 5MB ed il peso di oltre
una tonnellata, equipaggiava il computer
IBM 305 RAMAC



L'Hard Disk più piccolo-
Toshiba 2004 – 2007
32 mm × 24 mm × 5 mm
Capienza: 2, 4, 8 GB
ormai soppiantato da SSD e MicroSD

<https://obsoletemedia.org/0-85-inch-hard-disk/>
<https://www.global.toshiba/ww/news/corporate/2004/01/pr0801.html>

Approfondimenti



Martin Davis - Il calcolatore universale - Da Leibniz a Turing

<https://www.adelphi.it/libro/9788845927416>

Approfondimenti (IA)

